The Journal of Open Source Software

# Bracing the SWAP hydrological model for the future with the pySWAP Python package.

**Mateusz Zawadzki** [1¶], **Mark van de Brink** [2], **and Marijke Huysmans** [1]

**1** Vrije Universiteit Brussel, Belgium **2** Wageningen University and Research, the Netherlands ¶ Corresponding author

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:**

**Submitted:** 28 November 2025
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

## Summary

pySWAP is an open-source Python package for creating, running, analyzing, and sharing Soil-Water-Plant-Atmosphere (SWAP) hydrological models. As a concept, the first version was presented during at EGU General Assembly in 2024 (Zawadzki, 2024). SWAP calculates crop water requirements and simulates interactions between water balance components such as evaporation and groundwater storage (Kroes et al., 2017). The primary purpose of pySWAP is to eliminate the need for plain-text input files that SWAP requires, enabling smoother automation and seamless integration of modeling workflows with the variety of analytical tools the Python ecosystem offers. It also provides modern and attractive documentation for SWAP modelers.

## Statement of Need

SWAP has played an important role in the advancement of agrohydrological research over the last 50 years and has been widely applied in studies focusing on agricultural water management and irrigation optimization (Heinen et al., 2024; Kroes et al., 2017). The model is particularly valuable in agricultural applications due to its integration with the dynamic crop growth model WOFOST (Boogaard et al., 2014) which enables the simulation of crop responses to varying water and nutrient conditions. SWAP is part of tools used for policy making in the European Union (e.g. FOCUSPEARL (Van Den Berg et al., 2019)) and the Netherlands (e.g. WaterVision Agriculture (Hack-ten Broeke et al., 2019)).

SWAP is written in Fortran and consists of modules and routines developed by different authors over 50 years. The code and compiler settings are open-source under the GNU General Public License. Version 4.2.0 is distributed with supplementary materials and documentation via swap.wur.nl. Like many Fortran programs, SWAP requires plain-text ASCII files with custom extensions. The TTUTIL library (Kraalingen & Rappoldt, 2000) parses these files and loads variables into memory. The program executes with given settings and writes output files before termination.

Although this workflow is sufficient for small projects, it becomes cumbersome when managing multiple models, especially when running them on high-performance computing systems. This is often necessary for sensitivity analysis, scenario analysis, or calibration procedures across multiple field sites. These issues are exacerbated by the growing SWAP community and increasing complexity of research questions hydrologists address using the model.

Tools for handling SWAP input/output files have been published only in R (e.g., rSWAP (Shore, 2023) and SWAPTools, distributed with SWAP 4.2.0). Prior to pySWAP, no structured, tested, and peer-reviewed Python library for SWAP existed. Opening this new interaction pathway could expand the user community and improve modeling quality through open feedback platforms.

DRAFT

# Package architecture and functionality

pySWAP's object-oriented design is inspired by flopy, a Python package for handling MODFLOW models (Bakker et al., 2025; Hughes et al., 2024), with functionality split into modules for intuitive access. Users primarily interact with the `components` and `model` modules. Each component (e.g., `meteorology`, `crop`) groups together co-dependent attributes, similar to sections of the original input files. After definition, components are grouped into a `Model`. Components and their mixins contain all functionality related to SWAP model definition, while `Model` and its parent classes are responsible for building and running the model and capturing results. Users also have access to validation mechanisms to detect incorrect model setups and utility functions for model definition, including reading standard SWAP input into Python objects and integration with external databases. Functionality like spatial support and sensitivity analysis is under development for future plugin integration.

## Modelling workflow

We recommend using the `pyswap` CLI to start new projects. Running `pyswap init` prompts users for project metadata and generates a well-structured directory with a default Jupyter notebook and pixi.toml file for dependency management. Users begin by defining all relevant `components`, including file classes (see code block below). Once defined, a `Model` object is created. When `Model.run()` is called, all configuration files are saved to a temporary directory and a `Result` object is created after reading the output files. The run status is printed to the console upon program termination.

```python
import pyswap as ps
from pyswap import testcase

# metfile_from_csv will return a MetFile object, a file class
meteo_data = ps.components.meteorology.metfile_from_csv(
    metfil="283.met", csv_path=testcase.get_path("hupselbrook", "met")
)

# Meteorology object is an example of a section class
meteo = ps.components.meteorology.Meteorology(
    lat=52.0,
    alt=21.0,
    swetr=0,
    metfile=meteo_data,
    swdivide=1,
    swmetdetail=0,
    altw=10.0,
    angstroma=0.25,
    angstromb=0.5,
)

# the rest of the model definition

ml = ps.Model(
    meteorology=meteo,
    # Other elements...
)
```

To get started with `pyswap`, users can consult the documentation's quick start section or preview the interactive Binder environment with documentation notebooks.

DRAFT

## Integrations

### WOFOST Crop Parameters Database

The package provides access to databases of calibrated crop parameters (Wit & Burggraaff, 2025). Users can inspect parameter content, modify them, and update their pyswap crop settings objects accordingly.

### HDF5 database

pyswap models and results can be stored in HDF5 format, a versatile scientific computing standard. Each model is saved (optionally with results) as a pickled object with metadata.

### Belgian and Dutch soil databases

Packages `dovwms` (Zawadzki, 2025a), `simplesoilprofile` (Zawadzki, 2025b), and DutchSoils (Brink, 2025) provide quick access to Belgian and Dutch soil databases, enabling workflow automation and reducing potential errors.

# Testing

The current version has 84% code coverage using the pytest testing framework. Tests target specific features (e.g., CLI functionality) and validate model results against known benchmarks within acceptable error margins.

# CI/CD

The package implements robust continuous integration and deployment. Package and documentation builds are tested before each release, and documentation is automatically deployed with each new software version.

# Research projects using pyswap

pyswap has been successfully used in two research projects, with publications in preparation:

- Project GROW - modeling soil moisture conditions at an experimental field where treated wastewater is reused for subirrigation, and
- WaterScape - determining the role of soil-water-plant-atmosphere interactions in shallow groundwater and rooting zones for local water availability and land-use practices.

# Directions of development

There is no such thing as "finished software" and feature creep is extremely hard to shake off. While pyswap is production-ready, we acknowledge open issues (tracked on GitHub) and continue active development. However, at this stage we wish to see what users consider as priorities. Beyond the software itself, we aim to provide a platform for community engagement to accelerate development.

Key future features we are currently developing as plugins around pyswap:

- spatial context support (netCDF integration),
- sensitivity analysis and parameter estimation integration,
- improved multicore processing and plotting,
- flexible validation mechanisms, and
- versioned documentation.

DRAFT

## Acknowledgments

The authors would like to thank all those who gave valuable feedback to this work at conferences and (face-to-face) meetings. Thanks to Ali Mehmandoostkotlar, Erika Lucia Rodriguez Lache and Sarah Garré from the Flanders Research Institute for Agriculture (ILVO) who supported the initiative at its early stages. The Python packages of Pastas (Collenteur et al., 2019) and flopy (Bakker et al., 2025) served as an inspiration for this project. Moreover, the authors would like to thank the broad SWAP users community for their suggestions, constructive criticism and, above all, their contributions to pyswap.

## Funding

This work has been funded by the Interdisciplinary Research Project funding, an internal grant awarded to interdisciplinary research teams at the Vrije Universiteit Brussel, and WaterScape, a project funded by the Dutch Research Council, which explores ways to transform the Dutch water system in response to climate change.

## References

Bakker, M., Post, V., Hughes, J. D., Langevin, C. D., White, J. T., Leaf, A. T., Paulinski, S. R., Bellino, J. C., Morway, E. D., Toews, M. W., Larsen, J. D., Fienen, M. N., Starn, J. J., Brakenhoff, D. A., & Bonelli, W. P. (2025). *Flopy v3.10.0.dev5*. U.S. Geological Survey Software Release. https://doi.org/10.5066/F7BK19FH

Boogaard, H. L., Wit, A. J. W. de, Roller, J. A. te, & Diepen, C. A. van. (2014). *WOFOST Control Centre 2.1 and WOFOST 7.1.7*. Alterra, Wageningen University & Research Centre.

Brink, M. van den. (2025). *DutchSoils: Python package to get soil data from the Dutch Soil Map, Staring series and BOFEK clustering*. https://github.com/markvdbrink/dutchsoils; GitHub.

Collenteur, R. A., Bakker, M., Caljé, R., Klop, S. A., & Schaars, F. (2019). Pastas: Open Source Software for the Analysis of Groundwater Time Series. *Groundwater*, *57*(6), 877–885. https://doi.org/10.1111/gwat.12925

Hack-ten Broeke, M. J. D., Mulder, H. M., Bartholomeus, R. P., Van Dam, J. C., Holshof, G., Hoving, I. E., Walvoort, D. J. J., Heinen, M., Kroes, J. G., Van Bakel, P. J. T., Supit, I., De Wit, A. J. W., & Ruijtenberg, R. (2019). Quantitative land evaluation implemented in Dutch water management. *Geoderma*, *338*, 536–545. https://doi.org/10.1016/j.geoderma.2018.11.002

Heinen, M., Mulder, M., Van Dam, J., Bartholomeus, R., De Jong Van Lier, Q., De Wit, J., De Wit, A., & Hack - Ten Broeke, M. (2024). SWAP 50 years: Advances in modelling soil-water-atmosphere-plant interactions. *Agricultural Water Management*, *298*, 108883. https://doi.org/10.1016/j.agwat.2024.108883

Hughes, J. D., Langevin, C. D., Paulinski, S. R., Larsen, J. D., & Brakenhoff, D. (2024). FloPy workflows for creating structured and unstructured MODFLOW models. *Groundwater*, *62*(1), 124–139. https://doi.org/10.1111/gwat.13327

Kraalingen, D. W. G. van, & Rappoldt, C. (2000). *Reference manual of the FORTRAN utility library TTUTIL v. 4* (Report No. 5). Plant Research International. https://swap.wur.nl/TTutil/427/Ttutil4_13.pdf

Kroes, J. G., Van Dam, J. C., Bartholomeus, R. P., Groenendijk, P., Heinen, M., Hendriks, R. F. A., Mulder, H. M., Supit, I., & Van Walsum, P. E. V. (2017). *SWAP version 4*.

DRAFT

Wageningen Environmental Research. https://doi.org/10.18174/416321

Shore, M. (2023). *Rswap - an r interface for the SWAP model*. https://doi.org/10.5281/zenodo.7795152

Van Den Berg, F., Tiktak, A., Van Kraalingen, D. W. G., & Boesten, J. T. T. I. (2019). *User manual for FOCUSPEARL: Version 5.5.5* (No. 164). WOT Natuur & Milieu. https://doi.org/10.18174/509006

Wit, A. de, & Burggraaff, O. (2025). *WOFOST_crop_parameters: Parameter sets for the WOFOST crop simulation model implemented in YAML*. GitHub. https://github.com/ajwdewit/WOFOST_crop_parameters

Zawadzki, M. (2024). pySWAP: Python wrapper for SWAP hydrological model. *EGU General Assembly Conference Abstracts*, 19745.

Zawadzki, M. (2025a). *dovwms: Python package to retrieve data from soil texture WMS layers from Databank Ondergrond Vlaanderen (DOV)*. https://github.com/zawadzkim/dovwms; GitHub. https://doi.org/10.5281/zenodo.17458122

Zawadzki, M. (2025b). *simplesoilprofile: Python package to work with soil profiles*. https://github.com/zawadzkim/simplesoilprofile; GitHub.

DRAFT